NAVI // CHROMIUM

the browser is part of the OS.

On navi, the browser is not just an app. It is an **application runtime**, a **policy surface**, a **visual layer**, and eventually a platform we intend to shape ourselves. The web is navi's ideal app layer — portable, inspectable, offline-capable when we build it that way, and already understood by the tools people use every day.

The stack is already real: **72 shipped webapp launchers**, a webapp CLI, first-launch defaults, and optional browser installers. Ownership is the next step — and the point was never Chromium for its own sake. It's a stable web runtime navi can integrate deeply *without taking choice away*.

One thing should be said plainly before any of the engineering below: **we love Chromium as a technology.** Not as a brand, not as any company's product — as an engine. It's the most capable open web runtime ever built: a fast renderer, a genuine app platform, portable across everything we care about. Every criticism on this page is aimed at a specific integration, never at the project itself. You don't write a page like this about something you don't respect — and Chromium is an excellent surfboard for the open web. We're riding it.



↓ take this page offline — the full report as a PDF

01 the webapp runtime
02 first-launch defaults
03 the browser lineup
04 edge on navi
05 upstream citizenship
06 blackice
07 security architecture
08 the horizon: accela
09 roadmap & guardrails
10 navicast

SHIPPED · THE WEBAPP SYSTEM

navi turns the web into first-class desktop software

A navi webapp is not a bookmark. It is an installed desktop entry with its own icon, its own window, and the same launcher visibility as a native application.

catalog	→	install	→	discover	→	run
wired/naviApps/		navi-webapp		rofi / app menu		chromium --app=…

The public CLI is `navi-webapp` — `install`, `uninstall`, `list`, `doctor`. The wrapper translates a stable user-facing command into the catalog backend; `doctor` checks launchers, icons, caches, and common install mistakes.

Why app mode matters: the service gets an independent window in the compositor. Rofi launches it by name and icon, not by URL. Task switching treats it as an application surface. And the underlying engine still receives Chromium policy, extensions, and security updates.

First-party front doors: navi radio, neighborli, glyyph, and the naviApps storefront make the strategy visible — the distro's own services arrive as web technology without feeling bolted on. Users add more from the 72-launcher catalog. The default install is curated rather than exhaustive; the wider catalog stays available on demand. Telegram and Element have moved toward native-app paths, which shows navi treats the web layer as a strong default mechanism rather than a dogma.

electron carries the desktop app story

Chromium isn't just the webapp runtime — it's the desktop app platform. When navi-radio's local music folder picker hit a wall (Brave disables the File System Access API by default, and a per-machine flag flip is not a shippable answer for a default app), the app didn't get a workaround — it became a native Electron app with real file dialogs. sumi followed for the same reason; FOSSfit came along too. The pattern is now the rule: web technology, trivially buildable, no browser API roulette.

The stack, stated plainly: TUI for terminal-native tools, `--app` windows for webapps, Electron for desktop apps. All three are Chromium under the hood — which is why the browser engine gets a page like this one. It's load-bearing infrastructure, not a product preference.

Why Chromium for the app layer: `--app` mode gives real application windows. The extension and configuration surfaces are the ones we can support consistently. And Electron means our desktop apps share the same engine as our webapps. One runtime contract, tested once, working everywhere. Choice of browser stays with the user; the app contract stays with the engine.

local-first engineering keeps the app layer dependable

navi's web stack includes pages that never need to become public internet services. A local page should work as a local page — including when Chromium is already running.

The failure that shaped the design: the naviApps storefront originally read its catalog with `fetch()`. Under `file://`, that required a file-access flag — and Chromium can ignore new command-line flags when an existing browser process handles the request. So the store could fail in the most ordinary case: the browser was already open.

The fix was architectural, not another launch flag. `apps.json` remains the source of truth, but `build-catalog.sh` emits an inlined `catalog.js`. The local page loads that data directly and no longer depends on file-origin fetch behavior.

Design rules that fell out of it:

- **Offline-first means no hidden server.** Local tools must work from the filesystem.
- **Existing browser state counts.** A clean-launch test is not enough.
- **Flags are part of the runtime contract.** All 72 launcher files currently carry both `--app` and `--force-dark-mode`.
- **Generated data must ship with its source.** Catalog changes update both `apps.json` and `catalog.js`.

Deterministic dark mode: Chromium's system-theme detection can be unreliable on sway when the desktop portal isn't answering. navi puts a wrapper at `/usr/local/bin/chromium` that calls Debian's browser with `--force-dark-mode`, and a generated desktop override routes menu launches through that wrapper without modifying Debian's packaged file. User control remains: because navi's flag comes first, an explicit later `--force-light-mode` can override it. A curated default is not the same as a lock.

↑ back to top

SHIPPED · FIRST-LAUNCH DEFAULTS

defaults arrive on first launch

navi ships a small, explicit set of browser defaults — not as managed enterprise policy, but per-user, through Chromium's `External Extensions` mechanism scoped to `~/.config/chromium`. The result is reproducible: a new profile still looks and behaves like navi.

- **nightshadeNeon** — the browser theme arrives on first launch. *Deployed.*
- **blackice** — xvoidsx's Manifest V3 ad and tracker blocker (our uBlock Origin Lite fork), installed by default. It replaced uBlock Origin Lite after the dogfood gate passed. *Deployed.*

The old `ExtensionInstallForcelist` managed policy was deliberately retired: `/etc/chromium/policies/managed` is shared with Chromium forks, so the force-install leaked into Helium with its "managed by your organization" banner. Per-user install keeps forks clean — and unlike force-install, the user can remove any of it. (The browser picker can still deploy policy to *other* Chromium-family browsers you choose, like Edge or Chrome; Brave, Chromium, and Helium are exempt.)

What this buys: coherence — theme and security defaults don't depend on profile sync. Privacy by default — blocking is present before anyone shops for an extension. Supportability — bug reports begin from a known baseline. Recovery — the baseline is reconstructed by redeployment, not by profile surgery.

What this obligates: keep the list small — a default carries responsibility even when it's removable. Pin stable channels only; no beta surprises. State ownership honestly: blackice is xvoidsx's fork of upstream's uBlock Origin Lite. Test browser coverage, because extension paths differ across Chromium-family packages. Coherence is useful only when the user can understand what's installed and why. And know when not to install at all: Brave gets nothing — Shields already covers blocking, and a password manager is never imposed (users bring their own).

↑ back to top

THE BROWSER LINEUP

one runtime contract, several browser choices

navi can be opinionated about the integration layer without pretending there is one correct browser for everyone.

- **Brave — the default.** Brave is the rare Chromium browser that treats privacy as the product: built-in ad and tracker blocking, fingerprinting resistance, no account

required. It ships as navi's default browser and runs navi's webapps, with navi's touch kept light — an ultradark theme, wide address bar, and compact tabs as user-overridable defaults, no managed policy, no force-installed extensions. It arrives already aligned with navi's direction instead of needing navi's policy layer to fix it first.

- **Chromium — the vanilla alternative.** Debian's Chromium ships alongside Brave. navi's wrapper, launcher overrides, app windows, and policy (nightshadeNeon theme + blackice) are designed around it and exercised by the default install.
- **Helium — also in navi-extras.** An open-source Chromium browser maintained by imput, based on ungoogled-chromium, with its own Linux packaging. Raven runs it as a daily driver. A choice, like the rest.
- **Microsoft Edge — a candidate option.** Being evaluated as a user choice, not a default. Its Linux build looks at home under nightshadeNeon and its performance controls are unusually granular. Its telemetry model doesn't match navi's zero-telemetry direction, so that tradeoff belongs in the recommendation rather than in fine print.
- **Google Chrome — the reference build.** When something renders oddly in a fork, Chrome is the control experiment: stock Chromium plus Google's services, straight from Google's own repository. Its telemetry model is Google's, which is exactly why it's a choice here and never the default. In navi-extras for anyone who needs the real thing.
- **Yandex Browser — the new arrival.** The newest addition to navi-extras, with its own take on the Chromium experience. Early days: the installer just landed, so this bullet grows up as we learn how it behaves on navi. Choice means choice — good tech is good tech, wherever it's from.
- **One menu, no surprises.** Chromium ships with the system; every browser above it installs through navi-extras. Installer scripts live under `scripts/installers/`, deploy to `/usr/share/navi/installers/`, and are never run by the main installer. One optional browser failure must not endanger the operating-system install.

Shipped: the browser picker. `navi-browser` detects installed Chromium-family browsers, runs a headless launch smoke test, and sets one canonical default in `~/.config/navi/default-browser` — Brave out of the box, changeable anytime. Webapp launchers resolve that setting at launch time instead of hardcoding `chromium` — every webapp runs under the chosen browser's `--app` mode. Switching browsers also offers to deploy navi's managed policy (theme, blackice) to the new browser's

policy directory, which the picker locates by probing each browser binary for its compiled-in policy path — except Brave, which is deliberately skipped: no managed policy, no force-installed extensions. The scope stays Chromium-based because app mode and managed-policy behavior are the contract navi can support consistently. Choice preserved, integration tested.

↑ back to top

MICROSOFT EDGE ON NAVI

edge has a real efficiency case — and visible linux debt

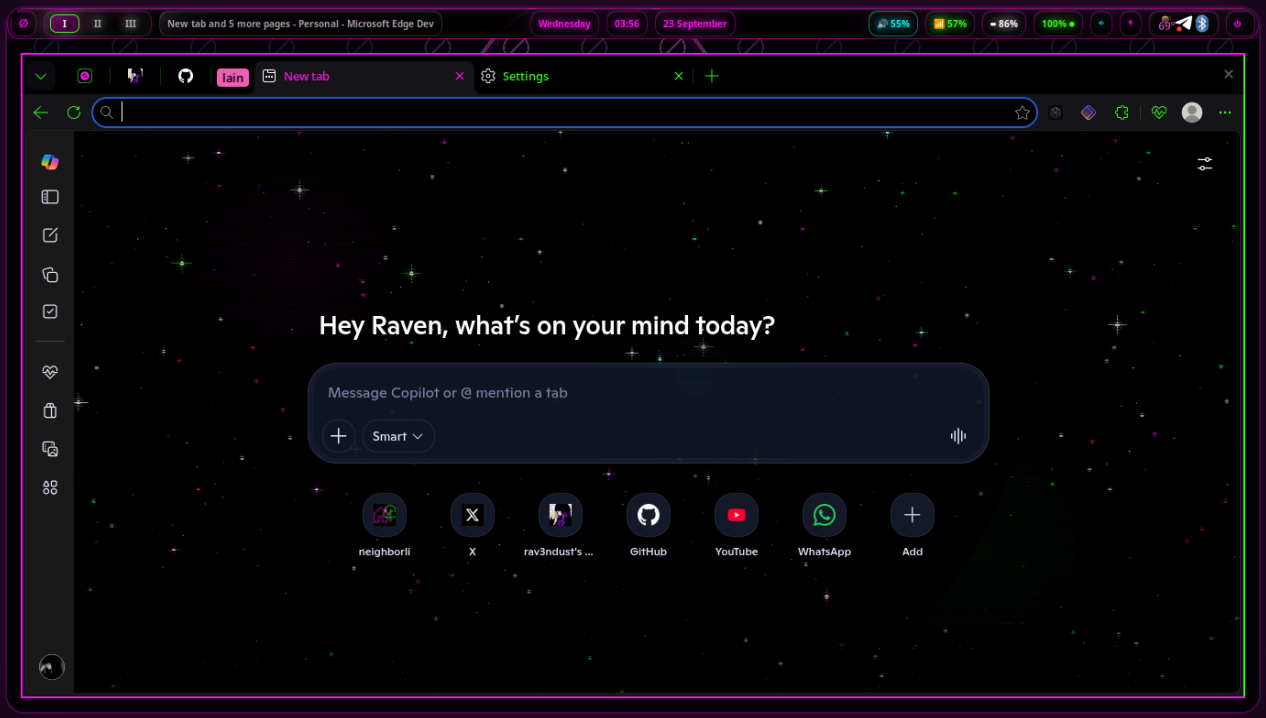


Figure 1. Microsoft Edge Dev on navi with nightshadeNeon applied. The fit is not hypothetical: Edge already participates in the distro's visual language.

Why it's worth supporting: Edge exposes sleeping-tab controls that put inactive tabs to sleep, let users choose the inactivity interval, and allow exceptions. Microsoft describes the feature as a way to reduce memory and CPU use. That is not a benchmark claim for navi hardware — it is a product capability with obvious relevance to low-power systems and machines with limited memory.

Why it's not the default: navi's direction is zero telemetry. Edge's product and account stack makes it a tradeoff rather than a clean philosophical fit. The honest position is stronger: users who value Edge's workflow or efficiency controls should be able to choose it, while navi states clearly why it remains optional.

The integration bar: visual fit is necessary but not sufficient. A browser option also needs dependable Linux window behavior, working first-party controls, predictable updates, policy compatibility, and app-mode launches that survive real daily use.

↑ back to top

UPSTREAM CITIZENSHIP

integration includes trying to make upstream linux support better

navi's first response to a browser bug should not always be a distro-only patch. When the failure belongs upstream, the useful work is evidence, a clear report, and verification when a fix lands.

tracker id	observed behavior	state
EDGE-LINUX-001	The Copilot toolbar button does not function on the tested Linux build.	Fixed in Dev channel (verified 11 Oct 2026); Stable pending
EDGE-LINUX-002	Enabling "Use system title bar and borders" produces no change; Edge's custom chrome remains in place.	Open; awaiting response

23 September 2026 — evidence sent upstream. Raven submitted both issues through Edge's built-in feedback system with a screenshot, a screen recording, and a detailed write-up, identifying himself as a navi maintainer who wants Edge to be a reliable distro option.

11 October 2026 — the first fix lands. About three weeks after the report, the Copilot button works in the Dev channel, verified on navi hardware. Whether our report

caused the fix is unknowable — Microsoft's internal notes aren't public. But a documented report from a distro maintainer went in on September 23rd, and a years-broken button started working about three weeks later. That's the whole story, and it's enough.

What's next: EDGE-LINUX-002 (the system title bar) is still open, and the Dev fix still needs verifying on Stable. The relationship is also growing in a second direction — blackice is headed for the Edge Add-ons store, desktop and (we hope) Android, so Edge users get our blocker without sideloading from Google's store. File their bugs, ship their users our work: that's the whole posture.

Now: a public record is maintained — `docs/edge-linux-feedback.md` keeps the issue IDs, exact behavior, submission history, and current status in one place. Both issues are also tracked with their status on our issues page, alongside every other bug we know about — ours and upstream's.

When fixed: verify on navi. The tracker calls for testing on eiri's Debian trixie base, against Edge Dev and Stable, on real navi hardware. A vendor response is not the same as a verified fix.

A distro maintainer lobbying Microsoft for better Linux support is unusual leverage. It should be spent deliberately: reproducible behavior, visual evidence, one public status page, no inflated claims. A private patch shifts maintenance onto navi and hides the Linux problem from everyone else; an upstream repair improves Edge for other distributions too. The same principle applies to anything navi might one day own — dogfood is due diligence. Daily use is how update lag, policy gaps, regressions, and browser-specific assumptions become visible before they're inherited.

↑ back to top

SHIPPED · PRIVACY LAYER

blackice moves blocking from a chosen extension toward owned infrastructure

shipped

accela

blackice default in Chromium → native adblock-rust component

The transition was staged on purpose — navi replaced a proven blocker only after its own fork earned the job. blackice graduated: it replaced uBlock Origin Lite as Chromium's default blocker and is live on the Chrome Web Store.

What blackice is: xvoidsx's GPLv3 fork of uBlock Origin Lite — a Manifest V3 ad and tracker blocker with a nightshadeNeon interface. The blocking engine, filter-list machinery, and Declarative Net Request conversion remain upstream work. Raymond Hill's copyright and the uBlock Origin contributors' notices stay intact; xvoidsx adds its copyright only to its own interface, branding, defaults, and genuine curated-list work.

Boundaries that protect credibility: never present blackice as an official uBlock project. Never run it beside uBO or uBOL — overlapping blockers fight over the same requests. Never show fake statistics, controls, or placeholder lists. Keep the Chromium build as the supported target — no other target gets claimed until it's packaged and tested.

One brand, two engines: the extension is the portable phase — it can run in Chromium-family browsers now and build recognition around one privacy surface. The native accela phase is different work: an adblock-rust component sits closer to the browser and avoids making the extension the permanent architectural ceiling. The name and user experience stay continuous even as the engine changes.

↑ back to top

CASE STUDY · SECURITY ARCHITECTURE

security is why this engine carries weight

navi bets its app layer on Chromium for a reason that has nothing to do with features: the security architecture. Every webapp window and every Electron app on navi runs inside one of the most battle-tested sandboxes in consumer software.

Site Isolation: every site renders in its own OS process, so a compromised renderer can't reach across into your bank tab — the mitigation that blunted Spectre-class attacks for the web. **The sandbox:** renderers run with stripped privileges, so

exploiting the parser isn't the same as owning the machine. Billions of users and one of the industry's largest bug bounties keep both honest, and the memory-safety push — Rust in new components, hardened allocators — keeps shrinking the classic bug classes.

What navi inherits, plainly: webapp windows get the full multi-process sandbox for free. Electron apps get Chromium's sandbox too, though Electron's defaults are looser than the browser's — a tradeoff we watch in every app we package. We don't audit Chromium ourselves; we ride the update cadence, with Debian's security-tracked Chromium underneath and upstream fixes flowing the way Debian ships them.

The honest boundary: no sandbox is a promise, only a cost multiplier. Chromium's is the most expensive target on the desktop web to exploit at scale — and that's the whole argument. navi's app layer stands on the engine attackers find least worth the effort.

and then there's the speed

Security is why the engine is trustworthy; speed is why it's pleasant. **Blink** is one of the fastest renderers ever shipped, and **V8** turned JavaScript from a scripting toy into a compiled language — hidden classes, inline caching, tiered JIT. That's the quiet reason navi's entire app layer can be web tech: the runtime isn't a compromise, it's the fastest horse. On the low-spec hardware navi loves, that matters more, not less.

↑ back to top

THE HORIZON · EXPERIMENTAL

the goal is a navi-native browser, not a logo swap

accela is where the browser becomes an owned interface to the wired: visual identity, privacy, search, agents, and the smallweb designed as one system. We're experimenting with different ways to make our own browser happen — no timeline on the horizon yet — but the direction is set, and everything on this page is the groundwork for it.



The accela mark, locked September 2026. The pink-and-green eclipse with the chrome slash — simple color blocking a browser tab can carry, and a slash no other browser on earth uses.

↑ back to top

ROADMAP AND GUARDRAILS

own more of the layer without outrunning the proof

The sequence matters. navi can deepen Chromium integration now while keeping accela on the horizon rather than turning it into the next-release bottleneck.

- **Now** — harden the shipped contract. Keep Chromium, the 72 app launchers, local catalog behavior, deterministic dark mode, and current policy supportable on real navi hardware.
- **Next** — make browser choice explicit. Build the settings TUI, a canonical browser setting, detection, app-mode tests, and clear fallback behavior.
- **In parallel** — push upstream. Track Edge Linux issues; verify vendor fixes instead of closing them on response alone.
- **Done** — blackice graduated. It replaced the uBOL entry and ships by default in Chromium.

- **Later** — begin accela in sequence. The roadmap places it after the desktop navi launch, navidroid, and eiri work: a strategic horizon, not the next checkbox.

Non-negotiable principles: additive, not purist — a strong default doesn't require purging alternatives. Choice preserved — users choose the browser while navi owns the compatibility contract. Zero telemetry — xvoidsx-owned layers collect nothing by default. Open models, open web — agent features should widen the web rather than trap it inside one provider.

Scope boundaries: Edge remains optional even if its Linux bugs are fixed. accela should not claim integrations before they exist.

Choice does not come from refusing to make decisions. It comes from making the integration legible, keeping alternatives available, and separating what ships from what is merely planned.

↑ back to top

THE HORIZON · NAVICAST

navicast: casting without the cloud

Chromium invites you to cast media to connected devices from any page — and privacy forks strip that component out, because it needs Google’s keys and proprietary bits. accela can’t ship Google’s cast cleanly, so the gap becomes the feature: navi’s own casting.

- **Open and LAN-local.** An xvoidsx casting protocol with local discovery, no account, no cloud, no telemetry. Media never leaves your network.
- **Every navi machine is both sender and receiver.** Cast a tab, a video, or music from the laptop to the machine behind the TV — or to another navi box entirely.
- **The Pi is the TV box.** A navi mini variant on HDMI becomes the always-on receiver: an old TV plus a computer you own becomes a smart TV that never gets end-of-lifed.

Honest scope: this is for your media, not theirs — DRM’d streaming services won’t certify an open receiver, so Netflix-style casting is out. Local media, tabs, and music are the tractable core. Early exploration, not a roadmap commitment — but the shape

fits: the browser’s cast affordance, pointed at your own hardware instead of Google’s cloud.

↑ back to top

CLOSING / REFERENCE

the browser is the user's agent on the web

navi treats that role as infrastructure worth understanding and, piece by piece, worth owning. Today's stack is deliberately practical: Debian Chromium, app-mode windows, local pages, Rofi launchers, per-user defaults, and optional alternatives. Tomorrow's stack is more ambitious: a user-selectable Chromium runtime, blackice as an owned privacy layer, and accela as the browser where navi's visual, smallweb, search, casting, and agent ideas meet.

The continuity matters more than the brand name. navi is not waiting for accela to begin integrating the browser well, and it is not using today's Chromium dependence as an excuse to avoid owning tomorrow's design.

- `wired/scripts/naviwebapp.sh` — public webapp CLI
- `wired/naviApps/` — catalog, launchers, icons, local store
- `wired/chromium/` — external extensions, theme, and dark-mode wrapper
- `scripts/installers/` — optional browser installers
- `docs/edge-linux-feedback.md` — public Edge issue tracker

github.com/xvoidsx/navi · github.com/xvoidsx/blackice · the full report as a PDF

Sources

1. xvoidsx/navi — repository and eiri working tree.
2. naviApps local-catalog commit — why catalog data is inlined.
3. Chrome Enterprise policy reference — force-installed extension behavior.
4. imputnet/helium and helium-linux — project, licensing, and Linux packaging.
5. Microsoft Edge performance features — sleeping tabs and configuration.

6. navi Edge-on-Linux feedback tracker — issue details and submission log.
7. xvoidsx/blackice — scope, attribution, license, and supported target.

Status note: "shipped", "experimental", and "planned" describe project state as of 11 October 2026. Planned items are not presented as shipped features. — *open models, open web*
